Algebraic World Model: From Exploration to Proofs
Supplementary Material

Anonymous authors
Paper under double-blind review

References to sections, figures, tables and equations without a letter refer to the main paper.

A Method details

This appendix gives the definitions behind §2: concepts and the gate (§2.2), the simplification test (§2.3), and the Lean statements, certificates and their transfer to the environment (§2.4).

A.1 Environment, concepts and the gate. Following Zhao et al. (2026), an environment is a deterministic, goal-directed special case of a partially observable Markov decision process (Kaelbling et al., 1998): states $\mathcal{S}$, actions $\mathcal{A}$, observations $\mathcal{O}$, a transition function $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, an observation function $\rho : \mathcal{S} \to \mathcal{O}$ and goal states $G \subseteq \mathcal{S}$. The agent sees $o_t = \rho(s_t)$ and receives a signal when a level is cleared. A transition is a triple $x = (o, a, o')$, and a game's transition corpus $\mathcal{D}$ is the set of distinct transitions across its runs, deduplicated by content hash.

A concept $c = (\sigma_c, \alpha_c, \gamma_c, E_c)$ has an optional executable scope σ_c, an assertion α_c returning TRUE, FALSE or NA, an optional predictor γ_c, and a non-empty evidence set E_c. When the scope rejects x the verdict $v_c(x)$ is NA, otherwise it is $\alpha_c(x)$; an exception, an invalid return value or a timeout gives ERR. A concept is scored only on transitions outside E_c. The gate of Eq. (1) is enforced on the action command: it refuses the next action while the latest settlement has a FALSE or ERR verdict on any $x \in \mathcal{D}$, including the concept's own evidence. Figure 1 shows one concept.

(a) One concept of lp85, as the agent wrote it during play

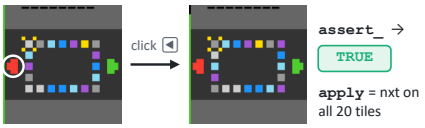
```
"""When the clicked arrow points against its belt's order,
   every tile color moves one place along the order."""
LEVEL    = "L1"                    # where it was first seen
SCOPE    = "clicks on a belt arrow; level change: NA"
EVIDENCE = ["4c0c8a2f…", "02bfcf1d…", …]  # 8 cited steps

def assert_(prev, action, nxt, ctx):       # checker
    order = belt(prev, action)
    if order is None: return "NA"
    a = [color(prev, p) for p in order]
    b = [color(nxt, p) for p in order]
    return "TRUE" if b == a[1:] + a[:1] else "FALSE"

def apply(prev, action, ctx):                # predictor
    order = belt(prev, action)
    if order is None: return None
    a = [color(prev, p) for p in order]
    return paint(order, a[1:] + a[:1])   # predicted cells
```

Abridged; docstring and scope translated, helper names shortened.

(b) On a recorded transition



1 Three verdicts, not two
A click off every belt arrow gives NA, not TRUE: "does not apply" is kept apart from "holds". A FALSE on any recorded step blocks the next action.

2 Not scored on its own evidence
It is scored only on the recorded steps it does not cite in EVIDENCE.

3 Both halves must match
apply's prediction is compared with the next frame, so a checker that passes while the predictor is wrong is caught.

Figure 1: A concept. (a) The concept of lp85 for a click on an arrow that points against its belt's order, abridged from the file the agent wrote during play. (b) On the first click of the certified run of level 1, its checker returns TRUE and its predictor reproduces the next frame on all 20 tiles. A FALSE on any recorded transition, or an exception counted as ERR, blocks the next action (Eq. (1)).

A.2 Objects, rules and the simplification test. A partial parser $\pi : \widetilde{\mathcal{O}} \rightharpoonup \mathcal{W}$ maps an observation, augmented by any explicitly supplied history or auxiliary information, to a world of typed objects; $\pi(o)$ abbreviates parsing with that context. Pixel interpretation is confined to the parser, and parse

failures are reported rather than guessed. A rule $r = (\sigma_r, \alpha_r)$ has a scope and an assertion on $(\pi(o), a, \pi(o'))$. The diagnostic set $\mathcal{N}$ of Eq. (2) contains at most 120 corrupted transitions per game, with a fixed seed. Each replaces part of a recorded transition (o, a, o') in one of five ways: the successor is o itself; the successor of the same action on another transition; the predecessor of another transition; o' with a few cells changed that do vary in the corpus; or another transition's o. We set

$$\mathrm{Rej}(r) = \{m \in \mathcal{N} : v_r(m) = \mathsf{FALSE}\}, \tag{A.1}$$

and a rule r is flagged as redundant on $\mathcal{N}$ when the other rules already reject what it rejects and the checks on the recorded transitions still pass,

$$\mathrm{Rej}(r) \subseteq \bigcup_{r' \neq r} \mathrm{Rej}(r'). \tag{A.2}$$

A field is redundant on the diagnostic corpus when erasing it to a constant changes no verdict on $\mathcal{D} \cup \mathcal{N}$. Figure 2 shows the test on `lp85`.

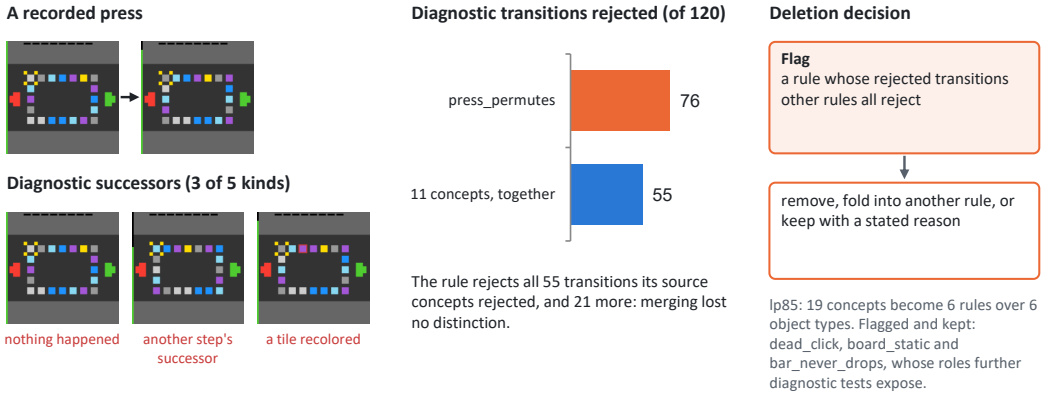


Figure 2: Diagnostic transitions, the merge check and deletion in `lp85`. Diagnostic transitions are built from recorded ones (three of the five constructions are shown on a real press). On the 120 diagnostic transitions $\mathcal{N}$ of this game, the merged rule `press_permutes` rejects all 55 that its eleven source concepts rejected together, and 21 more, so the merge lost no tested distinction; the whole book and the final six rules reject 117 and 119. A rule whose rejections the others cover is flagged and then removed, folded into another rule, or kept with a stated reason; the three flagged rules of `lp85`, `dead_click`, `board_static` and `bar_never_drops`, are kept because further diagnostic tests expose roles absent from this set.

A.3 Lean encoding, certificates and transfer. The step function dispatches among the rule definitions, returning a successor or a terminal outcome for actions admitted by the model's legality conditions. $\mathrm{everWon}(w, \bar{a})$ holds when some legal prefix of $\bar{a}$ reaches a winning outcome. The two statements of §2.4 are

$$\mathrm{Winnable}(w) :\Leftrightarrow \exists\, \bar{a}.\ \mathrm{everWon}(w, \bar{a}) = \mathsf{true}, \qquad \mathrm{Unsolvable}(w) :\Leftrightarrow \forall\, \bar{a}.\ \mathrm{everWon}(w, \bar{a}) = \mathsf{false}.$$

For level ℓ, the initial world w_0^ℓ is parsed from that level's first screen in the certified run, and the certified actions are $\bar{a}^\ell$. The reachability proof establishes $\mathrm{traceOk}(w_0^\ell, \bar{a}^\ell)$, i.e. that every computed state agrees with the recorded frame on the components the formal model represents, and $\mathrm{Winnable}(w_0^\ell)$ with $\bar{a}^\ell$ as witness. An impossibility proof establishes Unsolvable within a budget through a potential satisfying Eq. (3). Each theorem is closed by kernel reduction, its axioms lie within $\{$`propext`, `Classical.choice`, `Quot.sound`$\}$, and no proof uses `native_decide`; compiled with Lean 4.34.0, the published `Game.lean` of every game compiles and its `consistent` and `faithful` theorems depend only on `propext`.

The conclusions transfer to the environment under the following condition (Aineto & Scala, 2024; Clarke et al., 1994). Let $h : \mathcal{S} \to \mathcal{W}$ map concrete states, augmented with observation history where required, to modeled states.

Proposition A.1. If the model $\widehat{\mathcal{M}}$ over-approximates the environment, with matching initialization, every relevant real transition represented, real goals mapped to model goals, and cost and budget semantics that do not exclude a cheaper real execution, then

$$\text{Unsolvable}_{\widehat{\mathcal{M}}}\big(h(s_0)\big) \implies \forall\, \bar{a}.\, T^{\bar{a}}(s_0) \notin G.$$

Agreement with recorded transitions establishes this premise only on the transitions it covers, so AWM states its impossibility results as Unsolvable of the learned model.

B Twenty-five games

This appendix gives the per-game data behind §3, Table 1 and Table 2, the test of frozen rules on unseen transitions, and the `lp85` proof sizes of §2.5.

B.1 Per-game results. Table 1 gives AWM's results and model for each game; Table 2 gives the official scores of all systems.

Table 1: AWM on each game. *Score*: official score of the certified run. *Cert.*: actions in the certified clears, which the score counts. *Total*: every action AWM sent to the engine, including uncertified sessions, failed attempts and resets. *Human*: actions of the human baseline. *Concepts*: concepts written during play, with those withdrawn in brackets. *Rules (Lean)*: rules translated into Lean. *Replayed*: levels whose certified winning run Lean replays from the first screen to the modeled goal. *Thms*: Lean theorems. †Mechanisms taken from the game's public source code were given to the agent during play; the concepts they brought in do not count as discovered from play.

Game	Lv.	Score	Cert.	Total	Human	Concepts	Obj. types	Rules	Rules (Lean)	Replayed	Thms
`ar25`	8	100.0	253	253	748	17 (3)	6	8	5	8/8	43
`bp35`	9	100.0	374	374	651	26 (4)	3	11	10	9/9	41
`cd82`	6	100.0	70	661	171	44 (13)	7	11	10	6/6	81
`cn04`	6	100.0	261	745	789	22 (1)	4	9	8	6/6	51
`dc22`	6	100.0	1,037	2,664	1,228	48 (6)	8	17	14	6/6	77
`ft09`	6	100.0	80	168	208	15 (6)	6	7	4	6/6	50
`g50t`	7	100.0	288	811	879	20 (1)	10	13	11	7/7	64
`ka59`	7	97.6	421	4,061	730	22 (8)	9	11	10	7/7	67
`lf52`	10	100.0	905	905	1,339	47 (6)	12	11	10	10/10	64
`lp85`	8	100.0	79	200	388	23 (4)	6	6	4	8/8	47
`ls20`	7	100.0	368	1,467	776	22 (3)	12	21	18	7/7	42
`m0r0`	6	100.0	215	486	1,107	26 (5)	8	9	8	6/6	67
`r11l`	6	100.0	81	184	233	26 (4)	6	9	8	6/6	65
`re86`	8	100.0	547	2,153	1,255	40 (4)	7	12	8	8/8	42
`s5i5`	8	99.3	461	2,618	638	49 (9)	5	19	17	8/8	70
`sb26`	8	100.0	124	424	213	20 (1)	6	16	12	8/8	53
`sc25`	6	100.0	146	401	350	27 (1)	5	18	11	6/6	31
`sk48`	8	100.0	263	1,407	1,070	39 (8)	6	7	4	8/8	66
`sp80`†	6	100.0	138	371	518	58 (23)	3	10	6	6/6	25
`su15`†	9	100.0	90	904	361	37 (15)	5	16	12	9/9	64
`tn36`	7	100.0	125	292	317	33 (9)	5	7	4	7/7	28
`tr87`	6	100.0	119	430	414	13 (3)	6	7	4	6/6	50
`tu93`	9	100.0	187	566	462	17 (0)	8	10	8	9/9	76
`vc33`	7	100.0	167	538	447	14 (0)	9	10	7	7/7	75
`wa30`	9	100.0	649	1,521	1,843	37 (6)	8	8	7	9/9	35
Total	183	99.9	7,448	24,604	17,135	742 (143)	170	283	220	183/183	1,374

B.2 Frozen rules on unseen transitions. For `lp85` and `ft09` we freeze the concepts written during play (19 and 9) and the rules after Structure & Simplify (6 and 7), and run them on transitions from random presses (seeds 0–9), keeping only states whose screen never occurs in the recorded run. Each game has a set of new trajectories on the original levels (T1) and two modified levels, each checked by exhaustive search on the modified engine: in `lp85`, tile swap U moves one goal tile so that level 6 cannot be won and D adds one more swap that makes it winnable again; in `ft09`, pattern change FU replaces level 2's hidden click pattern with a plus shape, which makes it unwinnable, and FD adds a one-pixel change that makes it winnable again. A rule set accepts a true transition if no rule judges it false and rejects a wrong one, made by the benchmark's five corruption operators, if some rule judges it false.

Table 2: Official scores on the 25 public ARC-AGI-3 games. Baselines are the official ARC Prize results: Claude Opus 5 (High) and GPT-6 Astra (Max) under the standard harness, which provides no code execution, and GPT-6 Astra (Max) under its provider-adapter harness. AWM is scored on its certified run. baseline1 from its ARC Prize scorecard; Memento 3 as reported in (Zhao et al., 2026).

		Official score (max 100)					
Game	Lv.	Claude Opus 5	GPT-6 Astra	GPT-6 Astra (adapter)	baseline1 (Rodionov, 2026)	Memento 3 (Zhao et al., 2026)	**AWM (ours)**
ar25	8	100.0	100.0	100.0	100.0	100.0	**100.0**
bp35	9	0.0	2.2	100.0	100.0	100.0	**100.0**
cd82	6	0.6	82.9	100.0	100.0	100.0	**100.0**
cn04	6	47.6	100.0	100.0	100.0	100.0	**100.0**
dc22	6	44.8	100.0	100.0	100.0	100.0	**100.0**
ft09	6	100.0	100.0	100.0	100.0	100.0	**100.0**
g50t	7	0.0	0.0	100.0	100.0	100.0	**100.0**
ka59	7	10.4	75.0	100.0	100.0	100.0	**97.6**
lf52	10	1.8	10.9	100.0	100.0	100.0	**100.0**
lp85	8	100.0	100.0	100.0	100.0	100.0	**100.0**
ls20	7	5.4	83.5	100.0	100.0	100.0	**100.0**
m0r0	6	28.6	71.4	100.0	100.0	100.0	**100.0**
r11l	6	100.0	100.0	100.0	100.0	100.0	**100.0**
re86	8	58.3	77.8	100.0	100.0	100.0	**100.0**
s5i5	8	100.0	100.0	100.0	100.0	100.0	**99.3**
sb26	8	77.8	100.0	100.0	100.0	100.0	**100.0**
sc25	6	0.0	47.6	100.0	84.2	100.0	**100.0**
sk48	8	8.3	19.7	100.0	100.0	100.0	**100.0**
sp80	6	56.3	100.0	100.0	100.0	100.0	**100.0**
su15	9	10.4	27.4	100.0	100.0	100.0	**100.0**
tn36	7	6.0	53.6	100.0	90.1	100.0	**100.0**
tr87	6	47.6	100.0	100.0	100.0	100.0	**100.0**
tu93	9	3.6	3.1	100.0	100.0	100.0	**100.0**
vc33	7	98.8	73.3	100.0	100.0	100.0	**100.0**
wa30	9	10.6	80.0	100.0	100.0	100.0	**100.0**
Mean	183	40.7	68.3	100.0	99.0	100.0	**99.9**
Games at 100		5	10	25	23	25	**23**

Table 3: Frozen rules on unseen transitions, before / after Structure & Simplify (%). "Wrong rejected" is the range over the four corruption operators for which both rule sets can read both frames. On FU and FD every rejection comes from the rule that nothing else changes: each click changes 110 or 146 cells instead of 38.

Set	n	True accepted	Wrong rejected	Next frame predicted
lp85 T1 (new trajectories)	2,073	100 / 100	99.5–100 / 99.5–100	99.95 / 99.95
lp85 tile swap U	2,080	100 / 100	100 / 100	100 / 100
lp85 tile swap D	2,080	100 / 100	100 / 100	100 / 100
ft09 T1 (new trajectories)	2,026	99.0 / 100	99.0–100 / 100	—
ft09 pattern change FU	2,015	0 / 0	—	—
ft09 pattern change FD	2,016	0 / 0	—	—

B.3 `lp85`: the complete model and proof size. Figure 3 shows the complete model of `lp85`; its panels (d) and (e) are parts (b) and (c) of Figure 3. Claude Opus 5.5 proves each of 34 `lp85` tasks winnable (with a winning click sequence) or unwinnable (with a Lean proof), once with model A, the eleven concepts written during play, and once with model B, the single `press_permutes` rule; three runs per task and model, 204 runs in all, every verdict checked against exhaustive search on the engine. Table 4 gives the results.

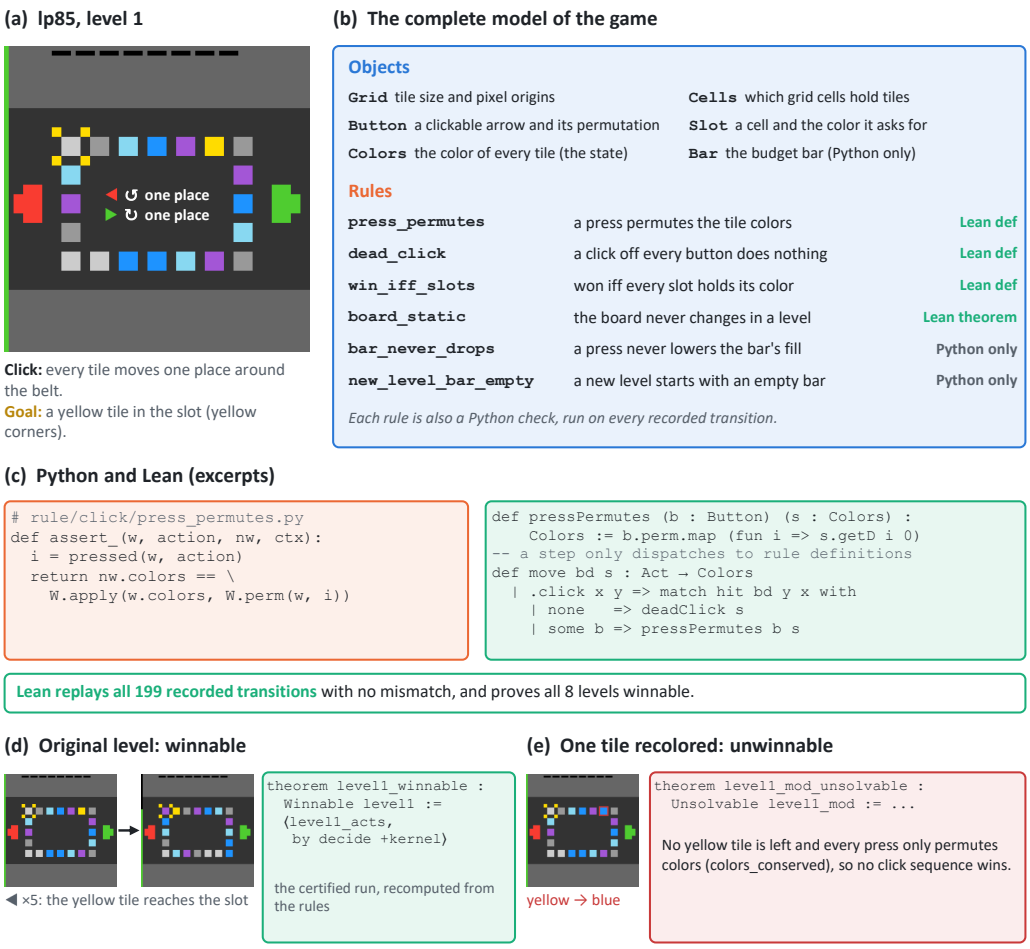


Figure 3: The complete algebraic world model of an ARC-AGI-3 game. (a) Level 1 of `lp85`: 20 colored tiles form a closed belt, and each click on an arrow moves every tile one place around the belt, counterclockwise for the left arrow and clockwise for the right; the level is won when a yellow tile stands in the slot marked by yellow corners. (b) All objects and rules of the model, with the form each takes in Lean; the two rules about the budget bar stay in Python. (c) Excerpts: one rule as its Python check, the same rule as a Lean definition, and the Lean step, which only dispatches to rule definitions. Lean replays every recorded transition of the game with no mismatch. (d) On the original level, Lean checks a winning witness, the certified run. (e) With the only yellow tile recolored, the conservation law of `press_permutes` shows that no click sequence wins. Both statements are theorems about the model.

C Sealed levels

This appendix gives the construction, protocol and per-run results behind §4 and Table 3, and the experiments of §4.2.

Table 4: Proving 34 `lp85` tasks with the eleven concepts (A) and the merged rule (B): runs proved, and medians of lemmas, proof lines and kernel time over the runs of each group. No run gives a wrong answer; A's three unproved runs are two timeouts and one run that stopped without an answer. On winnable instances both models are proved by a click sequence and `decide`.

Instances	Runs	Proved A / B	Lemmas A / B	Lines A / B	Kernel (s) A / B
original levels (winnable)	24	24 / 24	0 / 0	4.5 / 4	1.00 / 0.83
goal colour removed (unwinnable)	24	24 / 24	6 / 3	75.5 / 28	0.90 / 0.80
level 6 swaps (one each)	6	6 / 6	2 / 1.5	36.5 / 24	1.69 / 1.22
large layouts (winnable)	18	16 / 18	0 / 0	5.5 / 4	1.95 / 1.00
large layouts, altered (unwinnable)	18	17 / 18	12 / 4	117 / 49.5	2.82 / 2.41
ring spacing (unwinnable)	12	12 / 12	12 / 6.5	111 / 62.5	4.27 / 3.02
all	102	99 / 102			

C.1 Construction and engine ground truth. We locate the field that stores the budget and confirm that editing only it changes only the budget. We find $B^\star$, the cost of the cheapest win, by exhaustive search on the engine, and seal the level at $B^\star - 1$. The search is completed before any run starts. The search runs on the sealed engine itself, over all four opening facings and all 13 actions (four moves and nine panel cells) from every reachable configuration, 905,587 on level 4 and 1,423,609 on level 5. The counter is kept outside the state key, at its least value per configuration; the search covers all six writes to the counter in the source and its two checks against the cap. Clicks outside the nine panel cells, scanned over the whole 64×64 screen on 30,001 sampled frame classes per level, change none of the 13 actions' outcomes. Table 5 gives the result, and Figure 4 shows level 4.

Table 5: The two sealed levels. Costs are in counter units; "gross" adds back the fuel refund of 10. At the sealed cap no state wins from any opening; the optimal route wins on the unmodified engine at cap $B^\star$ and loses at $B^\star - 1$.

Level	Public cap	Sealed cap	$B^\star$ (gross)	Optimal route
4	35	18	19 (29)	27 actions
5	65	35	36 (46)	42 actions

C.2 Agents and protocol. Three configurations play each sealed level, three runs each. AWM is Claude Opus 5.5 with the AWM procedure, a generic Lean skeleton and tools for recording predictions and checking rules, but no game rules. Claude Opus 5.5 and GPT-6 Astra without AWM receive the task, the game interface and the ordinary tools of a coding agent. Each run has up to eight attempts on the sealed level, and an attempt ends at a game over. After each attempt the agent answers the interview of Appendix D, which records a verdict and its argument. Arenas carry opaque names, contain no ground-truth file, and start with cross-run memory cleared.

C.3 Per-run results. Table 6 lists the 18 runs of Table 3.

C.4 Repair. After the eight attempts, the two AWM runs and the four runs without AWM whose bounds are wrong each receive one round of feedback: the engine has a cheaper win than the run's analysis allows. No rule, cost or route is named. The AWM message adds a self-audit instruction (relax rules consistently with the record, test the disagreements in the game, revise, prove again); the message to runs without AWM states the same logic in plain language. Each run gets at most four further attempts. Table 7 gives the results.

C.5 Transfer to the engine. We repair a level-5 model learned by AWM by replacing each transition the engine refutes with the observed one and charging the winning move as the engine does. The repaired model gives a bound of 36, and its certificate holds on every engine transition within cap 35 from the opening that `set_level` produces: 17,194,487 non-terminal engine transitions, evaluated as 32,523 distinct abstract queries, with no violation.

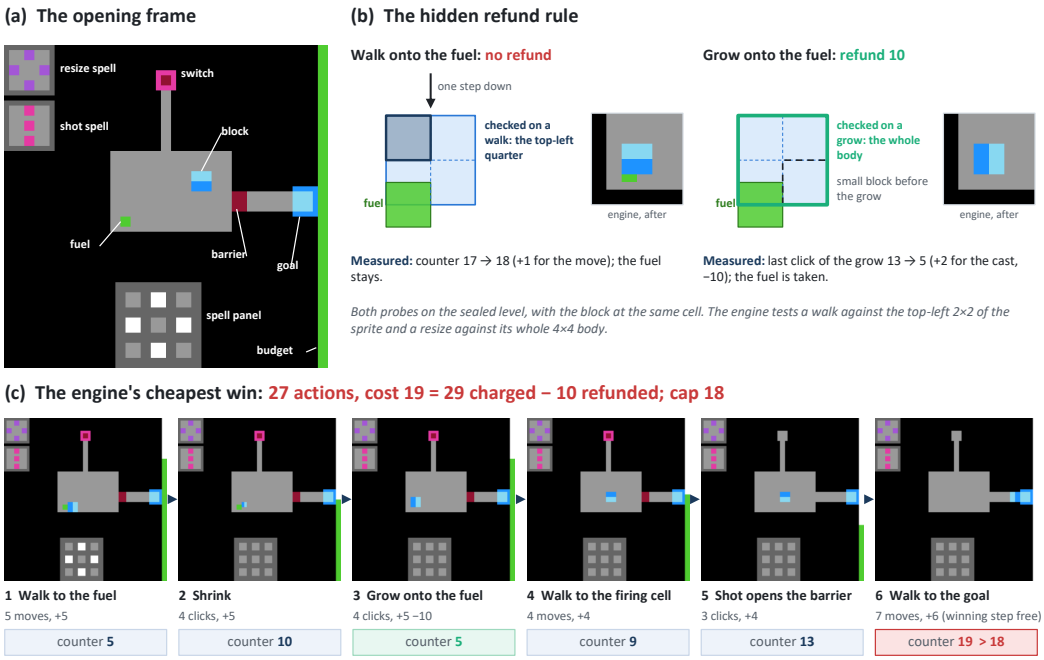


Figure 4: Sealed level 4 of `sc25`. (a) The opening frame. (b) The hidden refund rule: with the fuel under the bottom-left quarter of a large block, walking there pays nothing, because the engine checks only the top-left quarter, while growing there refunds 10; this is the rule the AWM run that proved 20 misread. (c) The engine's cheapest win, 27 actions at a cost of 19 against a cap of 18, with the counter after each stage. Frames are rendered from the real engine with the official palette; those in (c) are replayed on an otherwise identical level whose cap is 19, since on the sealed level the 26th action ends the game.

Table 6: The 18 runs of Table 3. *First claim*: the attempt of the first impossibility verdict. *Bound*: the least winning cost the run reports before feedback, in its own units for runs without AWM; $B^\star$ is 19 on level 4 and 36 on level 5. A bound is correct if it equals $B^\star$ or exceeds it only by charging the winning move, which the engine does not charge (37 on level 5; 42 actions against a budget of 41 in `fbac96`). *Lean proof*: a closed, kernel-checked impossibility theorem under the run's model.

Level	Agent	Run	First claim	Final verdict	Bound	Correct	Lean proof
4	AWM	`9ec8d4`	2	impossible	20	no	yes
4	AWM	`69fe87`	1	impossible	19	yes	yes
4	AWM	`2a3cc5`	2	impossible	19	yes	yes
4	Claude Opus 5.5	`2f1a1c`	—	undecided	—	no	no
4	Claude Opus 5.5	`d267df`	—	undecided	—	no	no
4	Claude Opus 5.5	`1dd6e6`	—	undecided	—	no	no
4	GPT-6 Astra	`44ec53`	8	undecided	22	no	no
4	GPT-6 Astra	`4d3b31`	8	impossible	22	no	no
4	GPT-6 Astra	`98893b`	—	undecided	—	no	no
5	AWM	`284967`	3	impossible	37	yes	yes
5	AWM	`4f4628`	2	impossible	36	yes	yes
5	AWM	`977714`	2	impossible	47	no	yes
5	Claude Opus 5.5	`339b0a`	4	impossible	47–49	no	no
5	Claude Opus 5.5	`202da3`	6	impossible	43	no	no
5	Claude Opus 5.5	`fbac96`	6	impossible	42 (budget 41)	yes	no
5	GPT-6 Astra	`2cdca5`	—	undecided	—	no	no
5	GPT-6 Astra	`3eb226`	—	undecided	—	no	no
5	GPT-6 Astra	`b3830e`	—	undecided	—	no	no

Table 7: The repair round of §4.3. Bounds of runs without AWM are in their own units, with the budget they state.

Run	Level	Attempts	Rule revised	Bound after	Lean proof
AWM `9ec8d4`	4	4	growing onto fuel collects it	19	yes
AWM `977714`	5	4	terrain, not range, stops the shot	37	yes
GPT-6 Astra `44ec53`, `4d3b31`	4	$\leq$4	—	19 (both)	no
Claude Opus 5.5 `339b0a`, `202da3`	5	$\leq$4	—	42 (budget $\sim$40), 68 (budget 64)	no

C.6 Locating an error from the proof. We inject an error that changes a single rule into the reference model of each sealed level, which matches the engine under the abstractions of §C.1; the main sample is the 14 injections on level 4 and 7 on level 5 that change the model's least winning cost, each run with seeds 1–3. Claude Opus 5.5 receives the faulty model with its proof and a probe service on the sealed engine, and is asked to find where the model and the engine disagree; C1 also receives a per-rule evidence table, C2 the same table and the proof's rule dependencies without the faulty rule named, and R is told which rule is faulty, while C0 probes at random without an agent and EX searches the engine exhaustively. A discovery is an observation in the scope of the faulty rule on which the engine disagrees with the model, counted in engine actions up to the first one, within a budget of 50 actions on level 4 and 5,000 on level 5.

Table 8: Locating an injected error (14 injections on level 4, 7 on level 5, three seeds each). *Found*: discovery within the action budget. *Median*: Kaplan–Meier median of actions to the first discovery, censored at the budget. *Named*: runs whose conflict log or final model points to the faulty rule. EX has no budget. The paired difference in discovery between C2 and C1 over the 63 sample–seed pairs is 0.238 (sample-clustered bootstrap 95% interval $[0.079, 0.397]$). On 12 error-free control runs C2 raises 0.00 false alarms per run and C1 0.33.

Arm	L4 found	L4 median	L5 found	L5 median	Found, both	Named	Tokens (median)
C0 random	12/42	> 50	0/21	> 5000	12/63	—	—
C1 per-rule evidence	16/42	> 50	1/21	> 5000	17/63	35/63	2.35M
C2 proof dependencies	29/42	27	3/21	> 5000	32/63	50/63	3.29M
R faulty rule named	30/42	26	5/21	> 5000	35/63	61/63	1.98M
EX exhaustive	14/14	136,852	1/7	—	15/21	—	—

C.7 Lean against a Python checker. On three models, the reference models of the two sealed levels (2,675 and 3,502 states) and the merged `1p85` model (869 states), we compare a Python search for the least winning budget (V1), a Python checker of the certificate conditions (V2) and the Lean development (V3), with 21 mutants per model: errors in the certificate generator (G), in the model definition (M1–M6), a change to only one of the Python and Lean versions (M7), and errors in the statement (S1–S7). Table 9 gives the results.

Table 9: Mutation test of three routes to the same bound, three models together. n/a: the route does not read the object that the mutant changes.

Mutant	V1 Python search	V2 Python checker	V3 Lean
generator error: invalid certificate rejected	n/a	12/12	12/12
of which the conclusion is false on the engine	n/a	9/9	9/9
model error (M1–M6) rejected	none	none	none
Python and Lean versions differ (M7) rejected	0/3	0/3	3/3
vacuous premises or empty start set (S1, S2) reported	0/6	0/6	6/6
reversed bound (S4) rejected	n/a	0/3	3/3
wrong goal set, other model version (S3, S7) rejected	none	none	none

D Interview prompts

This appendix reproduces verbatim the interview of Appendix C.2. After each attempt the harness sends questions 1–3, the formalization questions (AWM runs only) and the verdict lines, in the same session as the play; the level is then reset with the retry message. After the last attempt the final interview replaces questions 1–3 and keeps the same verdict lines.

Questions 1–3, every run.

```
That attempt failed. The game will reset this level, and you may get another attempt.

Answer the following. Keep it to what it needs; do not pad.

1. What happened on the level you were on when the attempt ended, and what do you think made
   it fail? Explain in plain prose, as if to someone who did not watch you play.

2. Do you still see a way to beat this level?
   - If you do: say what it is, and what you would still need to check to be sure of it.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. What will you do on the next attempt?
```

Formalization questions, AWM runs.

```
4. Before you answer this one, bring your formal system up to date. It is the thing you are
   building across attempts; the proof is downstream of it. Work in this order, in your
   working directory:

   a. Objects. What things are on the board, and what slots does each thing carry? For every
      object you keep, you should be able to name a pair of positions that stop being
      distinguishable if you drop it.
   b. Rules. Write each rule as a function: given a position and an action, what does it
      change? Each rule needs a domain you can actually execute.
   c. Simplify. Drop slots no rule reads. Merge rules that always decide alike. After this,
      your model's verdict on every frame you have observed must be unchanged.
   d. Formalize. Turn the objects and rules into Lean *definitions* - not theorems. Each rule
      ends up in exactly one of three states: a definition, a consequence of definitions, or
      explicitly marked "not in the model".

   If you already have a formal system from an earlier attempt, **amend it. Do not start a new
   one.** What you learned this attempt is a change to specific objects, rules or definitions;
   make those changes and leave the rest alone. A rewrite throws away the part of your work
   that was already right, and it destroys the record of what this attempt actually taught you.

5. Now prove your answer to question 2 **on that formal system**: that the level can be won,
   or that it cannot.

   - The proof must compile, and `#print axioms` on your main theorem must come back clean -
     no `sorryAx`, no `native_decide`. Report that output verbatim.
   - Say what your theorem actually quantifies over and what it assumes. If the proof needs a
     premise you did not establish, write it as an explicit hypothesis rather than burying it
     in a definition.
   - Name the rules from step 4b that your theorem leans on. If your Lean definitions carry
     something your rules do not - an extra assumption, a tighter domain, a step treated as
     atomic - say so here. A proof that is clean about the wrong system is worth nothing, and
     a model that is *stricter* than the game is the dangerous direction: "impossible in my
     model" does not transfer to the game.
   - Put the file in `lean/` and tell me its filename.
   - If you cannot complete the proof, say exactly where it breaks and what is missing. An
     honest "it does not go through, here is the gap" is worth more than a theorem that proves
     something weaker than it looks.

6. If this attempt taught you nothing that changes your formal system - no rule revised, no
   definition touched - do not invent a change. Say so instead, on its own line, in exactly
   this form, and then give the reason:

       FORMAL-UNCHANGED: <why this attempt added nothing to the model - what you tried, what
       you already knew, and what would have had to happen for the model to change>

   "No change, and here is why" is a real result. A change made only to look busy is not.
```

Verdict lines, AWM runs.

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | impossible_in_model_proved | impossible_in_model | undecided
CONFIDENCE: <an integer from 0 to 100>

The two "in_model" words are for when the level cannot be won under the rules you yourself
measured, and the step from "those rules" to "the real game" is an assumption you are stating
rather than something you established. Say which assumption, in your answer to question 2.
Use `impossible_in_model_proved` only if you are also handing over a Lean proof of it that
compiles and whose `#print axioms` is clean - name the file. Otherwise use `impossible_in_model`.
Your proof will be compiled and its axioms checked, so do not claim `_proved` on a file that
does not go through; `impossible_in_model` with an honest account of the gap is worth more.

Answer in writing only. Do not send any more game actions.
```

Verdict lines, baseline runs.

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | undecided
CONFIDENCE: <an integer from 0 to 100>

Answer in writing only. Do not send any more game actions.
```

Retry message.

```
You have another attempt on that level.

`POST /reset` restarts the level you were on; the levels you already beat stay beaten, and the
level's own counters start over. Play it again, as far as you can. When the attempt ends, stop
and report as before.
```

Final interview, questions.

```
That attempt failed.

Answer the following in writing. Give each answer the length it needs; do not pad.

1. What happened on the level you were on, and what do you think made it fail?
   Explain in plain prose, as if to someone who did not watch you play.

2. Do you still see a way to beat this level?
   - If you do: give the line of play, and say what you checked it against.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. Which rule did you newly work out across these attempts, and from which observation?

4. Which rule are you still unsure about, and unsure in what way?
```

References

Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.